STUDENT MANAGEMENT SYSTEM

WEEK 6 — PYTHON PROJECT REPORT

Student Name: Mirza Aliya Begum

Internship Domain: Python Development

Project Title: Student Management System

Programming Language: Python

Tools Used: Visual Studio Code, Git and GitHub

---

1. INTRODUCTION

The Student Management System is a Python-based application developed to manage student information efficiently. The project provides a simple menu-driven interface that allows users to add, view, search, update and delete student records.

The application also calculates student grades based on marks and displays statistics such as average marks, highest marks and lowest marks.

Student records are stored in a JSON file, allowing the application to retrieve saved information when it is run again.

This project was developed as part of the Week 6 final project for the Python internship.

2. PROBLEM STATEMENT

Managing student information manually can be time-consuming and may lead to mistakes, duplicate records and difficulty retrieving information.

The Student Management System aims to solve this problem by providing a simple application to organize student details, update records and calculate academic statistics.

3. PROJECT OBJECTIVES

The main objectives of the project are:

- To develop a functional Python application.
- To store and manage student information.
- To add, view, search, update and delete student records.
- To calculate grades based on student marks.
- To calculate average, highest and lowest marks.
- To save records using file handling.
- To handle invalid inputs and common errors.
- To apply object-oriented programming concepts.
- To gain practical experience with Git and GitHub.

4. PROJECT FEATURES

4.1 Add Student

Allows users to enter a student's ID, name, age and marks.

4.2 View Students

Displays all student records stored in the application.

4.3 Search Student

Searches for a student using the student ID.

4.4 Update Student

Allows users to modify existing student details.

4.5 Delete Student

Deletes a selected student record after confirmation.

4.6 Grade Calculation

Automatically calculates grades according to the student's marks.

4.7 Student Statistics

Displays the total number of students, average marks, highest scorer and lowest scorer.

4.8 File Storage

Stores student information in a JSON file so records remain available after restarting the application.

5. TECHNOLOGIES USED

Python: Used to develop the application logic.

JSON: Used to store student records in a structured format.

Visual Studio Code: Used to write, edit and execute the Python program.

Git: Used for version control and tracking project changes.

GitHub: Used to host the project repository online.

Microsoft Word: Used to prepare the project report.

6. PYTHON CONCEPTS IMPLEMENTED

6.1 Variables and Data Types

Variables store student IDs, names, ages and marks. Strings, integers and floating-point numbers are used.

6.2 Input and Output

The input() function accepts user information, while print() displays menus and student records.

6.3 Conditional Statements

If, elif and else statements validate inputs, select menu operations and calculate grades.

6.4 Loops

A while loop repeatedly displays the main menu until the user chooses to exit. For loops process student records.

6.5 Functions

Functions organize the program into reusable operations, including adding, viewing, searching, updating and deleting records.

6.6 Data Structures

A list stores student objects, and dictionaries organize student information before it is saved to JSON.

6.7 File Handling

The open() function reads and writes student records in the students.json file.

6.8 Exception Handling

Try and except blocks handle invalid numerical input and file-related errors.

6.9 Object-Oriented Programming

The project uses classes and objects to organize student information and application functionality.

6.10 Constructors

The **init**() method initializes student objects and management classes.

6.11 Inheritance

StudentManager inherits from RecordManager and reuses its student collection.

6.12 Encapsulation

The Student class uses a private marks attribute and a property setter to validate marks.

6.13 Operators and Type Casting

Comparison operators validate marks and ages. The int() and float() functions convert input into numerical values.

7. PROJECT WORKFLOW

The application follows these steps:

1. The program starts and loads previously saved student records.
2. A menu is displayed to the user.
3. The user selects an operation.
4. The program accepts the required input.
5. Input validation checks the supplied information.
6. The selected operation is performed.
7. Updated records are saved to the JSON file.
8. The menu appears again until the user exits.

8. IMPLEMENTATION DETAILS

The project contains three classes.

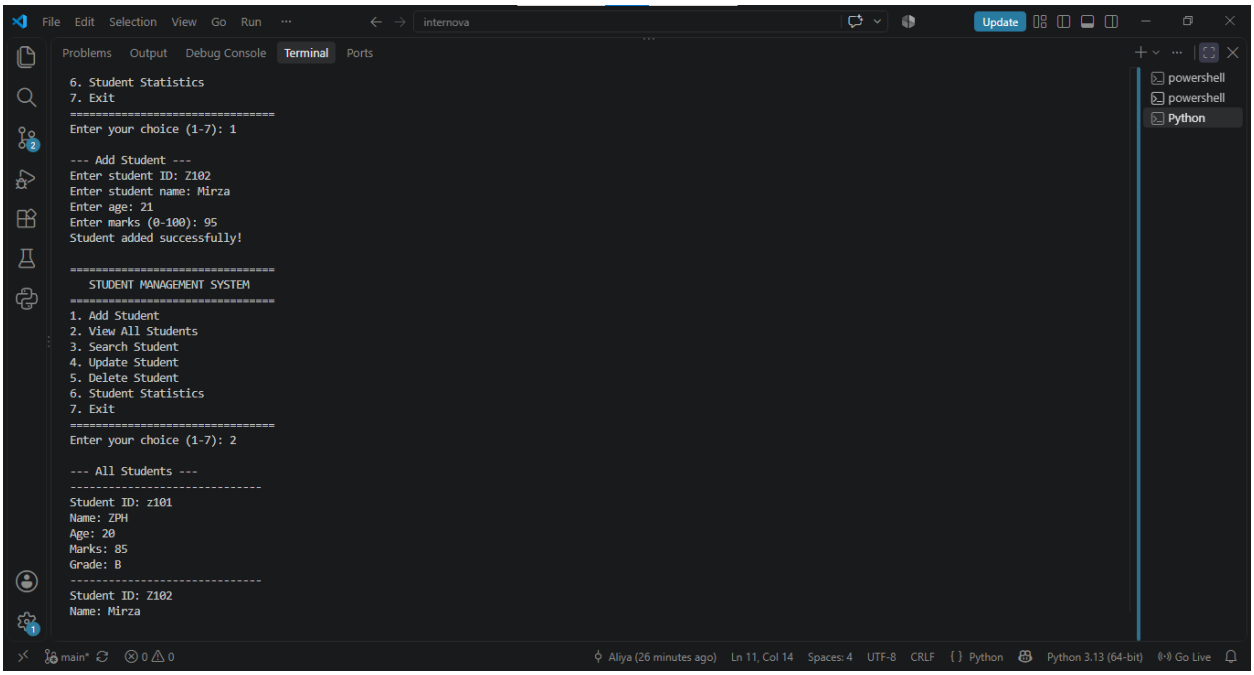
Student: Stores student details, validates marks, calculates grades and displays information.

RecordManager: Provides the basic student record collection.

StudentManager: Inherits from RecordManager and implements record management, file handling and statistics.

The main() function controls the application menu and calls the appropriate methods based on the user's selection.

9. SCREENSHOTS AND OUTPUT



10. SAMPLE TESTING

The application was tested using the following cases.

Test Case	Expected Result
Add a student	Student record is saved
View students	Student details are displayed
Search an existing ID	Matching student is displayed
Search an invalid ID	Student not found message appears
Update student details	Record is updated
Delete a nonexistent ID	Student not found message appears
Calculate statistics	Student statistics are displayed
Exit the application	Program terminates normally

During testing, a student with ID z101, name abc, age 20 and marks 85 was added successfully. The application displayed grade B. The student's name was subsequently updated to ZPH, and the statistics feature correctly displayed average marks of 85.00.

11. CHALLENGES FACED

The main challenges during development included:

- Validating student marks within the range of 0 to 100.
- Handling invalid numerical input.
- Preventing duplicate student IDs.
- Saving and loading student records.
- Updating and deleting the correct student record.
- Organizing the program using classes and functions.

These challenges helped improve my understanding of Python programming and problem-solving.

12. FUTURE IMPROVEMENTS

The application can be improved in the future by:

- Developing a graphical user interface.
- Connecting the application to an SQLite database.
- Adding login and authentication.
- Generating PDF and CSV reports.
- Adding attendance management.
- Supporting subject-wise marks and results.

13. CONCLUSION

The Student Management System demonstrates the practical application of Python programming concepts in a real-world style project.

The application successfully supports student record management, grade calculation, file storage, input validation and student statistics.

Developing this project improved my understanding of functions, loops, data structures, file handling, exception handling and object-oriented programming.

The project also provides an opportunity to learn version control and online project management using Git and GitHub.

14. GITHUB REPOSITORY

GitHub Repository URL: https://github.com/Aliya214/Student-Management-System.git